

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_videodataset_path = kagglehub.dataset_download('rehmang110/videodataset')
rehmang110_videosdataa_path = kagglehub.dataset_download('rehmang110/videosdataa')

print('Data source import complete.')
```

```
!pip install ultralytics
!git clone https://github.com/ifzhang/ByteTrack.git
# Replace deprecated np.float with float after cloning
%cd ByteTrack
!pip install -r requirements.txt
!python setup.py develop
!pip install cython
!pip install 'git+https://github.com/cocodataset/cocoapi.git#subdirectory=PythonAPI'
!pip install cython_bbox
!pip install onnxruntime
!pip install loguru
!pip install lap
%cd /kaggle/working/ByteTrack
```

[Show hidden output](#)

```
import torch
import numpy as np
if not hasattr(np, 'float'):
    np.float = float

import pandas as pd
from pathlib import Path
import sys
from IPython.display import Video
import shutil
import cv2
from ultralytics import YOLO
from IPython.display import Video

from collections import defaultdict
from yolox.tracker.byte_tracker import BYTETracker # From ifzhang/ByteTrack
from yolox.tracking_utils.timer import Timer
```

Creating new Ultralytics Settings v0.0.6 file 
 View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
 Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/>

```
print("PyTorch version:", torch.__version__)
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)

# pathlib is part of the Python standard library
print("Python version (for pathlib):", sys.version)
```

```
PyTorch version: 2.6.0+cu124
NumPy version: 1.26.4
Pandas version: 2.2.3
Python version (for pathlib): 3.11.13 (main, Jun 4 2025, 08:57:29) [GCC 11.4.0]
```

```

if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16
else :
    device = "cpu"
    dtype = torch.float32

print("Using device:", device)
print("Using dtype : " , dtype)

```

```

Using device: cpu
Using dtype : torch.float32

```

```

%cd /kaggle/working
source_path = Path("/kaggle/input/videosdataa")
dest_path = Path("/kaggle/working/inputsvideos")
dest_path.mkdir(exist_ok=True , parents=True)
for file in source_path.iterdir():
    shutil.copy(file , dest_path / file.name)

```

```

/kaggle/working

```

```

# Define argument class for ByteTrack
class ByteTrackArgs:
    track_thresh = 0.5
    track_buffer = 30
    match_thresh = 0.8
    aspect_ratio_thresh = 10.0
    min_box_area = 1.0
    mot20 = False

```

```

!ffmpeg -i /kaggle/working/video.mp4 -vf "scale=640:-1" -c:a copy /kaggle/working/video_resized.mp4

```

[Show hidden output](#)

```

video_path = Path("/kaggle/working/inputsvideos/gate1.mp4")

```

```

Video(str(video_path), embed=True)

```

[Show hidden output](#)

```

model = YOLO("yolov8n.pt")
model.to(device)
class_names = model.names

```

```

cap = cv2.VideoCapture(video_path)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
output_path = Path('/kaggle/working/output_video2.mp4')
out = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc(*'mp4v'), fps, (width, height))

```

```

tracker = BYTETracker(ByteTrackArgs)
unique_ids = set()
frame_id = 0

```

```

# Class names from YOLO (make sure they match your model)
CLASS_NAMES = model.model.names # Ultralytics YOLOv8
id_to_class = dict() # track_id -> class_name
class_counts = defaultdict(int) # class_name -> count

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame)[0]

```

```

detections = []

for box in results.bboxes:
    x1, y1, x2, y2 = map(float, box.xyxy[0])
    conf = float(box.conf[0])
    class_id = int(box.cls[0])
    if conf > 0.1:
        detections.append([x1, y1, x2, y2, conf, class_id]) # Add class_id

if len(detections) > 0:
    dets_tensor = torch.tensor([d[:5] for d in detections]) # [x1, y1, x2, y2, conf]
    class_ids = [int(d[5]) for d in detections]

    online_targets = tracker.update(dets_tensor.cpu(), (height, width), (height, width))

    for i, t in enumerate(online_targets):
        tlwh = t.tlwh
        track_id = t.track_id
        x, y, w, h = map(int, tlwh)

        # Assign class from detection (best guess: match by index)
        class_id = class_ids[i] if i < len(class_ids) else -1
        class_name = CLASS_NAMES[class_id] if class_id in range(len(CLASS_NAMES)) else "others"

        # Count if this is the first time seeing this track_id
        if track_id not in unique_ids:
            unique_ids.add(track_id)
            id_to_class[track_id] = class_name
            class_counts[class_name] += 1

        # Draw box + ID + class
        label = f"{class_name} ID:{track_id}"
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(frame, label, (x, y - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    # Draw summary on the left side of the frame
    x_offset, y_offset = 10, 20
    cv2.putText(frame, "🟢 Detected Objects:", (x_offset, y_offset),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
    y_offset += 25

    for cls_name, count in class_counts.items():
        cv2.putText(frame, f"{cls_name}: {count}", (x_offset, y_offset),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
        y_offset += 25

    out.write(frame)
    print(f"Frame {frame_id}: {len(unique_ids)} unique objects tracked.")
    frame_id += 1

cap.release()
out.release()

print(f"\n✅ Video saved as output_tracked.mp4")
print(f"📊 Total unique objects tracked: {len(unique_ids)}")
print(f"📊 Object class counts:")
for k, v in class_counts.items():
    print(f" - {k}: {v}")

```

[Show hidden output](#)

```

video_path2 = Path("/kaggle/working/output_video.mp4")
# do not now why not showing here but th output video is good i checked by downloaod it
Video(str(video_path2), embed=True)

```

